

UNIVERSITÉ MOULAY ISMAÏL
ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS — ENSAM
MEKNÈS

Filière : Intelligence Artificielle et Technologies des Données — Systèmes Industriels (IATD-SI)

SoutenanceAI

Plateforme web de gestion et de notation
intelligente des soutenances académiques

Rapport de projet — module **Digital Web Prototyping**

Réalisé par :

DJERI-ALASSANI Oubenoupou
2^e année cycle ingénieur

Encadré par :

Pr. BAKKAS B.

Résumé

L'évaluation des soutenances académiques repose aujourd'hui sur un processus entièrement manuel : prise de notes au vol, grilles papier, questions improvisées, absence de trace exploitable a posteriori. Ce projet propose **SoutenanceAI**, une application web complète développée avec le framework **Django**, dans laquelle une intelligence artificielle examinatrice assiste le professeur sur l'ensemble du cycle de vie d'une soutenance : transcription de la présentation orale en temps réel (Whisper), génération et lecture à voix haute de questions ancrées dans le contenu réel de l'étudiant (Claude), analyse comportementale dans le navigateur (contact visuel, expressions faciales, prosodie), puis notation multi-critères justifiée que le professeur valide, ajuste et exporte en PDF ou Excel.

L'application met en œuvre les concepts centraux du module *Digital Web Prototyping* : architecture **MVT** de Django, ORM et migrations, formulaires et validation, authentification multi-rôles, internationalisation (5 langues dont l'arabe en RTL), ainsi que des extensions avancées : WebSockets avec Django Channels (ASGI), pipeline d'agents IA configurables sur deux axes de personnalité, et une suite de **258 tests automatisés**.

Mots-clés : Django, MVT, WebSocket, Channels, IA générative, Claude, Whisper, notation automatique, soutenance, il8n.

Table des matières

Résumé	1
Introduction générale	5
1 Contexte, problématique et objectifs	6
1.1 Le processus traditionnel de soutenance	6
1.2 Problématique	6
1.3 Objectifs du projet	6
1.4 Lien avec le module Digital Web Prototyping	7
2 Analyse et conception	8
2.1 Acteurs du système	8
2.2 Besoins fonctionnels	9
2.3 Besoins non fonctionnels	9
2.4 Diagramme de cas d'utilisation	10
2.5 Diagramme de classes	10
2.6 Diagramme de séquence : déroulement complet d'une soutenance	12
3 Choix technologiques et architecture	15
3.1 Django et le pattern MVT	15
3.2 ORM, migrations et intégrité	15
3.3 Temps réel : ASGI, Channels et WebSockets	16
3.4 La couche IA	17
3.4.1 Transcription — Whisper sur Groq	17
3.4.2 Évaluation — Claude et le double axe de personnalité	17
3.4.3 Analyse comportementale — dans le navigateur	18
3.5 Internationalisation et expérience utilisateur	18
3.6 Architecture générale	19
4 Réalisation	20
4.1 Organisation du code	20
4.2 Authentification et parcours d'inscription	20
4.3 Classes, soutenances et partage par code	20
4.4 Planification des passages	21
4.5 La salle de soutenance	21
4.6 Le pipeline de notation	21
4.7 Suivi live et validation par le professeur	23
4.8 Emails automatiques	23
5 Tests, sécurité et qualité	24
5.1 Stratégie de tests	24
5.2 Sécurité	25
5.3 Qualité et maintenabilité	26

Conclusion et perspectives	27
Webographie	29
Annexe : captures d'écran	30

Table des figures

2.1	Diagramme de cas d'utilisation global de SoutenanceAI	10
2.2	Diagramme de classes du domaine métier	11
2.3	Automate d'états du champ statut de PassageEtudiant	12
2.4	Diagramme de séquence (1/2) : préparation et présentation orale avec trans- cription continue	13
2.5	Diagramme de séquence (2/2) : questions/réponses, pipeline de notation et validation	14
3.1	Architecture générale : clients, serveur ASGI, services externes	19
5.1	Page d'accueil publique : parcours professeur et étudiant, code de classe, sé- lecteur de langue et thème	30
5.2	Auto-inscription professeur (formulaire avec validation)	31
5.3	Dashboard professeur : classes avec code d'accès, nombre d'inscrits et soutenances	31
5.4	Détail d'une classe : étudiants inscrits et soutenances	32
5.5	Planification des passages : inscrits, création manuelle, auto-planification et planning exportable	32
5.6	Dashboard étudiant : classes, soutenances et état du passage	33
5.7	Salle de soutenance : slides (PDF.js), webcam étudiant, avatar du jury IA, timer et contrôles	33

Introduction générale

La soutenance est l'exercice académique par excellence : l'étudiant y défend un travail devant un jury qui doit, en quelques dizaines de minutes, évaluer à la fois le fond (rigueur, maîtrise du sujet) et la forme (clarté, posture, gestion du stress, qualité des réponses). Or ce processus souffre de limites bien connues : la charge cognitive du jury est élevée (écouter, questionner et noter simultanément), la traçabilité est faible (aucune transcription, aucune mesure objective), et l'équité entre candidats dépend de la fatigue et de la subjectivité du moment.

Parallèlement, trois familles de technologies ont atteint une maturité suffisante pour répondre à ces limites : les grands modèles de langage capables d'évaluer un discours selon une grille de critères en justifiant chaque note ; la reconnaissance vocale de haute qualité en quasi temps réel ; et l'analyse comportementale embarquée dans le navigateur, sans installation côté client.

SoutenanceAI est née de la rencontre de ces deux constats. Il s'agit d'une application web complète — et non d'un prototype jetable — développée avec Django dans le cadre du module *Digital Web Prototyping*. Elle couvre le cycle complet : création de classes et de soutenances par le professeur, inscription des étudiants par code d'accès, planification automatique des passages, salle de soutenance interactive de type visioconférence, notation IA multi-critères, validation par le professeur et production des livrables (rapports PDF, exports Excel).

Ce rapport est organisé en cinq chapitres. Le chapitre 1 précise le contexte, la problématique et les objectifs. Le chapitre 2 présente l'analyse et la conception : acteurs, besoins, **diagramme de cas d'utilisation**, **diagramme de classes** et **diagramme de séquence** détaillant le fonctionnement interne. Le chapitre 3 justifie les choix technologiques et l'architecture, en insistant sur le pattern MVT de Django. Le chapitre 4 décrit la réalisation fonctionnalité par fonctionnalité, extraits de code à l'appui. Le chapitre 5 traite des tests, de la sécurité et de la qualité logicielle.

Chapitre 1

Contexte, problématique et objectifs

1.1 Le processus traditionnel de soutenance

Dans une école d'ingénieurs, une session de soutenances mobilise un ou plusieurs enseignants pendant des journées entières. Pour chaque passage, le jury doit : accueillir le candidat, suivre la présentation en prenant des notes, préparer des questions pertinentes pendant l'écoute, les poser, évaluer les réponses, remplir une grille de notation, puis enchaîner immédiatement avec le candidat suivant. Les artefacts produits (notes manuscrites, grilles) sont rarement numérisés et jamais reliés à un enregistrement de la prestation.

1.2 Problématique

Le projet répond à la question suivante : *comment concevoir une application web qui automatise les tâches mécaniques d'une soutenance (transcription, chronométrage, questions de premier niveau, pré-notation documentée) tout en laissant au professeur la décision finale, et ce dans un cadre multilingue, équitable et traçable ?*

Cette question se décompose en sous-problèmes techniques précis :

- a) capter l'audio et la vidéo du candidat *dans le navigateur*, sans installation, et transcrire la parole en continu ;
- b) faire dialoguer en temps réel l'étudiant, le serveur et le professeur (transcription live, notes qui apparaissent instantanément) — ce que le cycle requête/réponse HTTP classique ne permet pas ;
- c) construire un *contexte d'évaluation* complet pour l'IA : transcription, contenu des slides et du rapport, questions/réponses, mesures comportementales, consignes du professeur ;
- d) garantir que l'IA note selon la personnalité pédagogique choisie par le professeur, avec un garde-fou pour les étudiants en situation de stress ;
- e) assurer une isolation stricte des données entre professeurs et un contrôle d'accès par rôle sur chaque vue et chaque WebSocket.

1.3 Objectifs du projet

- **O1** — Offrir au professeur une gestion complète : classes avec code d'accès et QR, soutenances configurables, planification manuelle et automatique des passages, exports.
- **O2** — Offrir à l'étudiant une salle de soutenance immersive : slides, webcam, jury IA vocal, questions interactives.
- **O3** — Produire une notation IA *justifiée* critère par critère, modifiable par le professeur, avec rapport PDF.

- **O4** — Fonctionner en 5 langues (fr, en, ar, es, de) avec support RTL complet pour l'arabe.
- **O5** — Atteindre une qualité logicielle vérifiable : suite de tests automatisés, contrôle d'accès systématique, secrets hors du code.

1.4 Lien avec le module Digital Web Prototyping

Le module porte sur la conception et le prototypage d'applications web avec Django. Le projet exploite la quasi-totalité des notions du cours et les prolonge :

Notion du cours	Mise en œuvre dans SoutenanceAI
Architecture MVC / MVT	4 applications Django découpées par domaine ; séparation stricte modèles / vues / templates (§3.1)
ORM et migrations	7 modèles métier, relations 1-N et N-N, propriétés calculées, 9 migrations versionnées
URLs et vues	Routage par <code>include()</code> par application, vues fonctionnelles décorées
Templates	Héritage (<code>base.html</code>), tags <code>i18n</code> , filtres personnalisés (<code>templatetags/extras.py</code>)
Formulaires	<code>ModelForm</code> , <code>Form</code> , validation personnalisée (<code>clean_*</code>), formsets dynamiques côté client
Authentification	<code>AbstractUser</code> étendu, 3 rôles, décorateurs d'accès maison
Fichiers statiques / médias	Upload PPTX/PDF/vidéo, conversion serveur, service des médias
Internationalisation	5 langues, <code>LocaleMiddleware</code> , RTL, compilation polib
<i>Extension</i> : ASGI	Django Channels, deux consumers WebSocket, Daphne
<i>Extension</i> : IA	SDK Anthropic et Groq, agents configurables, pipeline de notation
<i>Extension</i> : tests	pytest-django, pytest-asyncio — 258 tests

TABLE 1.1 – Correspondance entre le contenu du module et le projet

Chapitre 2

Analyse et conception

2.1 Acteurs du système

Superadmin

Administre la plateforme : création/suppression des comptes professeurs, statistiques globales, accès en lecture à toutes les soutenances.

Professeur

S'inscrit librement. Crée ses classes et soutenances, importe/inscrit ses étudiants, planifie les passages, suit les soutenances en direct, valide les notes, exporte les livrables.

Étudiant

Rejoint une classe via un code. Dépose ses fichiers, passe sa soutenance dans la salle interactive, consulte ses résultats.

Système IA (acteur secondaire)

Ensemble des services externes sollicités par le serveur : Whisper/Groq (transcription), Claude/Anthropic (questions, évaluation des réponses, notation, analyse vidéo et GitHub).

2.2 Besoins fonctionnels

Réf.	Acteur	Besoin
BF-01	Professeur	S'inscrire en autonomie et se connecter
BF-02	Professeur	Créer / éditer / supprimer une classe ; partager code, lien et QR
BF-03	Professeur	Créer une soutenance : langue, durées, critères pondérés, exigences (rapport, démo vidéo, dépôt GitHub), styles IA
BF-04	Professeur	Planifier les passages manuellement ou automatiquement (monôme/binôme/groupe, ordre, pauses) ; importer des groupes ; exporter le planning Excel
BF-05	Professeur	Suivre un passage en direct (transcription, statut, notes) et poser une question relayée vocalement
BF-06	Professeur	Valider / ajuster chaque note IA, re-déclencher la notation, exporter notes Excel et rapport PDF
BF-07	Étudiant	Rejoindre une classe par code avec création de compte intégrée (identifiant choisi)
BF-08	Étudiant	Déposer PPTX (converti et affiché en slides), rapport PDF, démo vidéo, URL GitHub
BF-09	Étudiant	Passer la soutenance : timer, micro, transcription continue, questions IA vocales, réponses à la voix
BF-10	Étudiant	Recevoir un rappel email 10 minutes avant son passage
BF-11	Étudiant	Consulter notes, synthèse, transcription et enregistrement
BF-12	Superadmin	Gérer les comptes professeurs et visualiser les statistiques
BF-13	Système	Transcrire l'audio par tranches de 30 s ; analyser le comportement ; noter chaque critère sur 20 avec justification

TABLE 2.1 – Besoins fonctionnels principaux

2.3 Besoins non fonctionnels

- **Temps réel** : latence de diffusion de la transcription < 2 s après la fin d'un chunk audio (WebSocket, pas de polling).
- **Sécurité** : contrôle d'accès par rôle sur 100 % des vues ; isolation par propriétaire (un professeur ne voit jamais les données d'un autre) ; secrets exclusivement en variables d'environnement.
- **Internationalisation** : interface intégralement traduite en 5 langues, dont une langue RTL ; langue de soutenance indépendante de la langue d'interface.
- **Accessibilité / UX** : thème clair/sombre persistant, icônes vectorielles uniformes, identité visuelle unique (variables CSS).
- **Testabilité** : logique IA isolée dans des services purs, mockables ; suite pytest exécutable en local sans clé API.
- **Portabilité** : développement sous Windows, SQLite par défaut, bascule PostgreSQL/-Redis par simple variable d'environnement.

2.4 Diagramme de cas d'utilisation

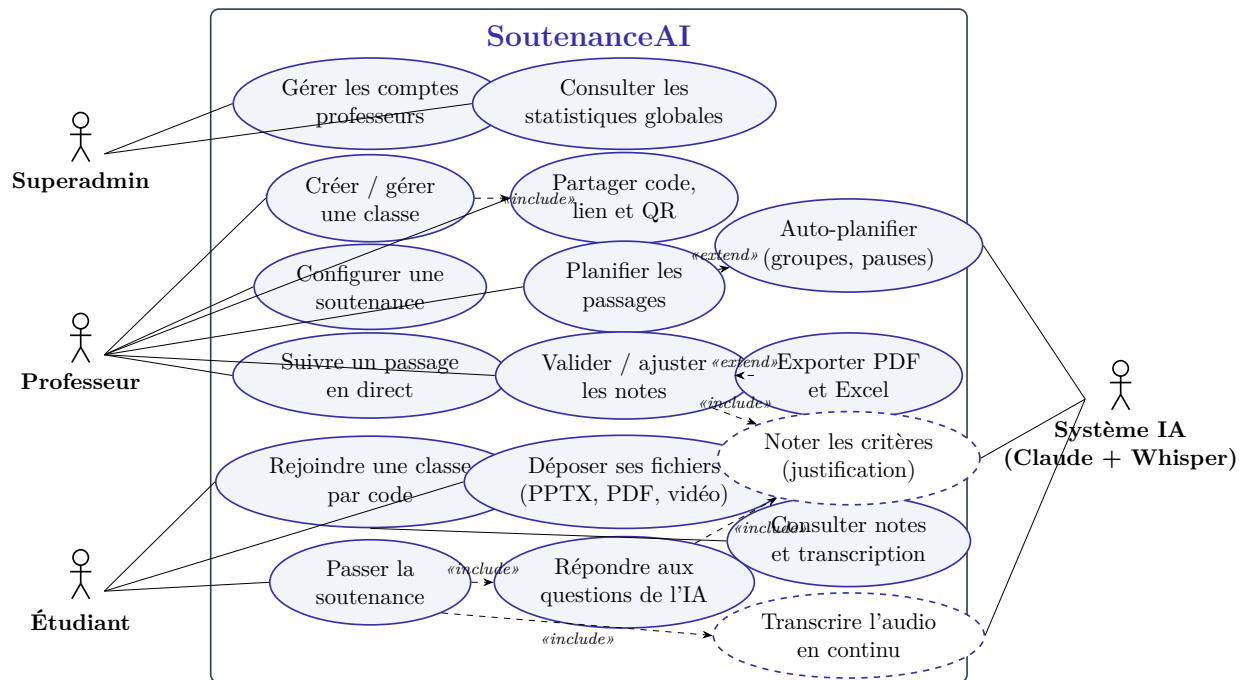


FIGURE 2.1 – Diagramme de cas d'utilisation global de SoutenanceAI

Le cas central est *Passer la soutenance* : il inclut la transcription continue et la session de questions/réponses avec l'IA. Côté professeur, *Valider les notes* inclut la notation IA préalable (le professeur ne part jamais d'une page blanche) et peut s'étendre vers les exports.

2.5 Diagramme de classes

Le modèle de données compte sept classes métier réparties dans trois applications (**accounts**, **sessions_app**, **notation**). La figure 2.2 présente les attributs significatifs et les multiplicités.

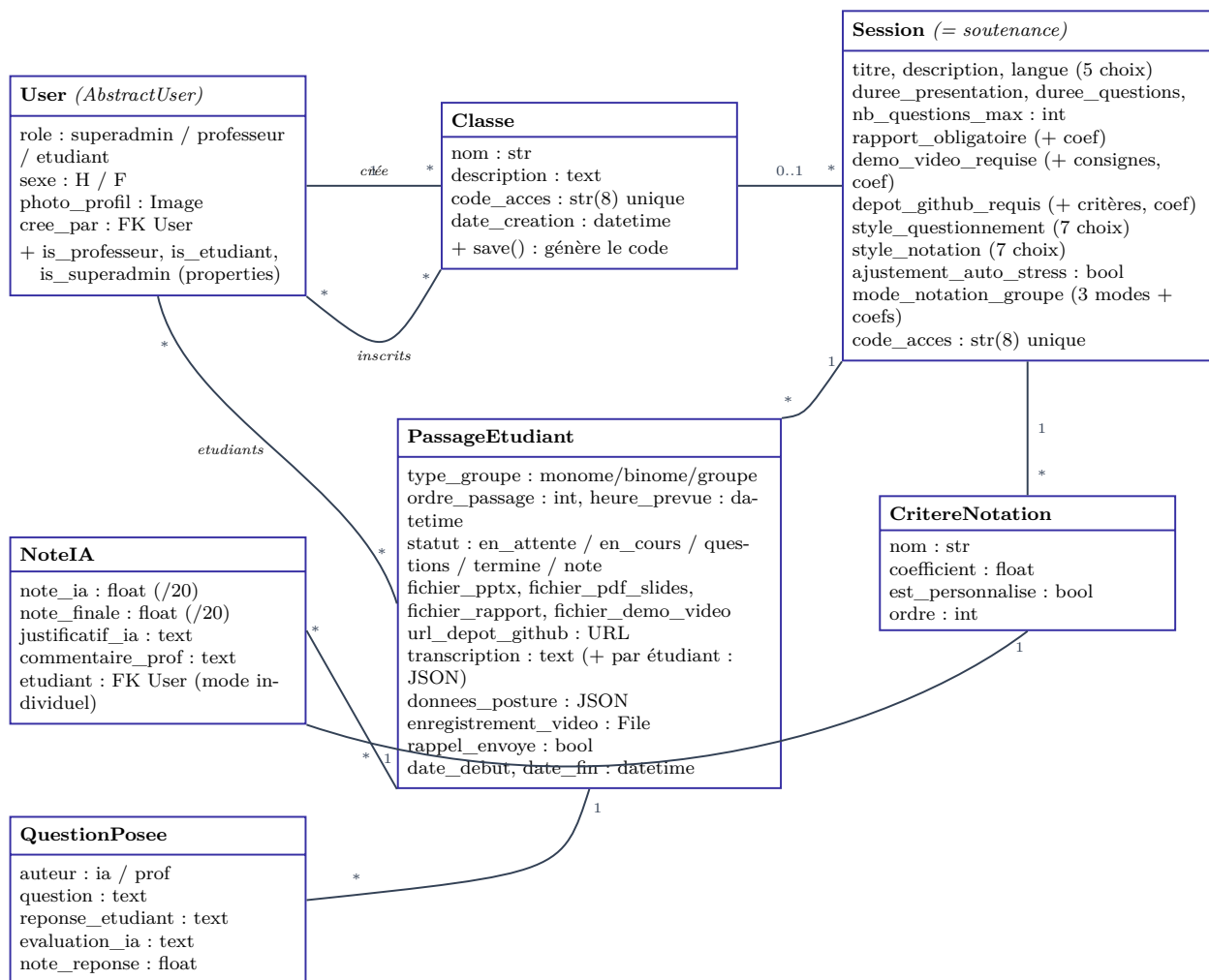


FIGURE 2.2 – Diagramme de classes du domaine métier

Points de conception notables.

- **User** étend **AbstractUser** avec un champ **role** discriminant — un seul modèle d'utilisateur, trois visages (`AUTH_USER_MODEL = 'accounts.User'`). Le champ **cree_par** relie chaque étudiant au professeur qui a créé son compte, fondement de l'isolation des données.
- **Classe** matérialise le concept « Google Classroom » : les étudiants la rejoignent *une fois*; toutes les soutenances de la classe leur sont ensuite accessibles.
- **PassageEtudiant** porte une relation N-N vers **User** : un passage peut être un monôme, un binôme ou un groupe ; la transcription individuelle par étudiant (JSON, alimentée par l'identification faciale) permet la notation individuelle au sein d'un groupe.
- **NoteIA** sépare **note_ia** (proposition de l'IA, immuable) et **note_finale** (décision du professeur) : la traçabilité de l'ajustement humain est conservée.

Justification des arbitrages de conception

Un schéma de données est une suite de compromis ; les principaux arbitrages du projet, avec les alternatives écartées, sont les suivants.

Relation N–N **PassageEtudiant** ↔ **User**, sans table intermédiaire explicite.

Une table d'association enrichie (de type **Participation**) n'a de valeur que si la relation

porte des attributs propres. Ici, les données « par étudiant dans le passage » sont la transcription individuelle et les notes : les premières vivent dans un JSON dédié, les secondes dans `NoteIA.etudiant`. Le `ManyToManyField` natif suffit donc, et garde l'API ORM simple (`passage.etudiants.all()`). Si un jour la relation devait porter un état (présence, retard...), une migration vers `through=` resterait possible sans changer l'API.

Transcriptions individuelles en JSON plutôt qu'en table dédiée.

Les chunks transcrits arrivent toutes les 30 s sur le WebSocket : une table dédiée imposerait une écriture indexée par chunk, pour des données qui ne sont jamais interrogées finement — elles sont lues *en bloc* une seule fois, au moment de la notation. Le champ `JSONField` (clé = id étudiant, valeur = texte cumulé) offre l'écriture append la moins coûteuse et évite une jointure de plus dans le pipeline. Contrepartie assumée : pas de requête SQL sur le contenu des transcriptions individuelles — besoin qui n'existe pas aujourd'hui.

QuestionPosee sans relation vers CritereNotation.

Les questions de l'IA sont générées à partir du *contenu* de la présentation, pas de la grille : les rattacher a priori à un critère serait arbitraire. Le rattachement se fait en aval : la session Q&R intégrale est réinjectée dans le contexte de notation, et c'est l'IA évaluatrice qui mobilise les réponses pour noter le critère « Réponses aux questions » — avec justification textuelle. On évite ainsi une classification artificielle question → critère.

Codes d'accès de 8 caractères plutôt qu'UUID dans les URL.

Le code doit être dicté à l'oral ou écrit au tableau. Un alphabet {A-Z, 0-9} sur 8 positions offre $36^8 \approx 2,8 \times 10^{12}$ combinaisons, tirées via le module `secrets` (cryptographique) avec re-tirage en cas de collision : illisible à deviner, trivial à partager.

Notation de groupe : trois modes plutôt qu'un.

Le mode unique « note identique » (le plus simple) a été écarté car il pénalise les groupes hétérogènes. Le mode individuel pur exige de savoir *qui parle* — d'où l'identification faciale. Le mode mixte pondéré ($c_g \times$ note collective + $c_i \times$ note individuelle) couvre le cas pédagogique le plus fréquent ; les coefficients sont des champs du modèle, pas des constantes.

Cycle de vie d'un passage. Le champ `statut` suit un automate strict, piloté par les vues et diffusé en temps réel au professeur :



FIGURE 2.3 – Automate d'états du champ `statut` de `PassageEtudiant`

2.6 Diagramme de séquence : déroulement complet d'une soutenance

Ce diagramme est le cœur de la conception : il décrit, de bout en bout, ce qui se produit entre le navigateur de l'étudiant, le serveur Django (vues HTTP et consumers WebSocket), les services IA externes et le navigateur du professeur, depuis l'ouverture de la salle jusqu'à la publication des notes.

La lecture s'organise en deux temps : la figure 2.4 couvre la préparation et la présentation orale (transcription continue) ; la figure 2.5 couvre la session de questions, le pipeline de notation et la validation.

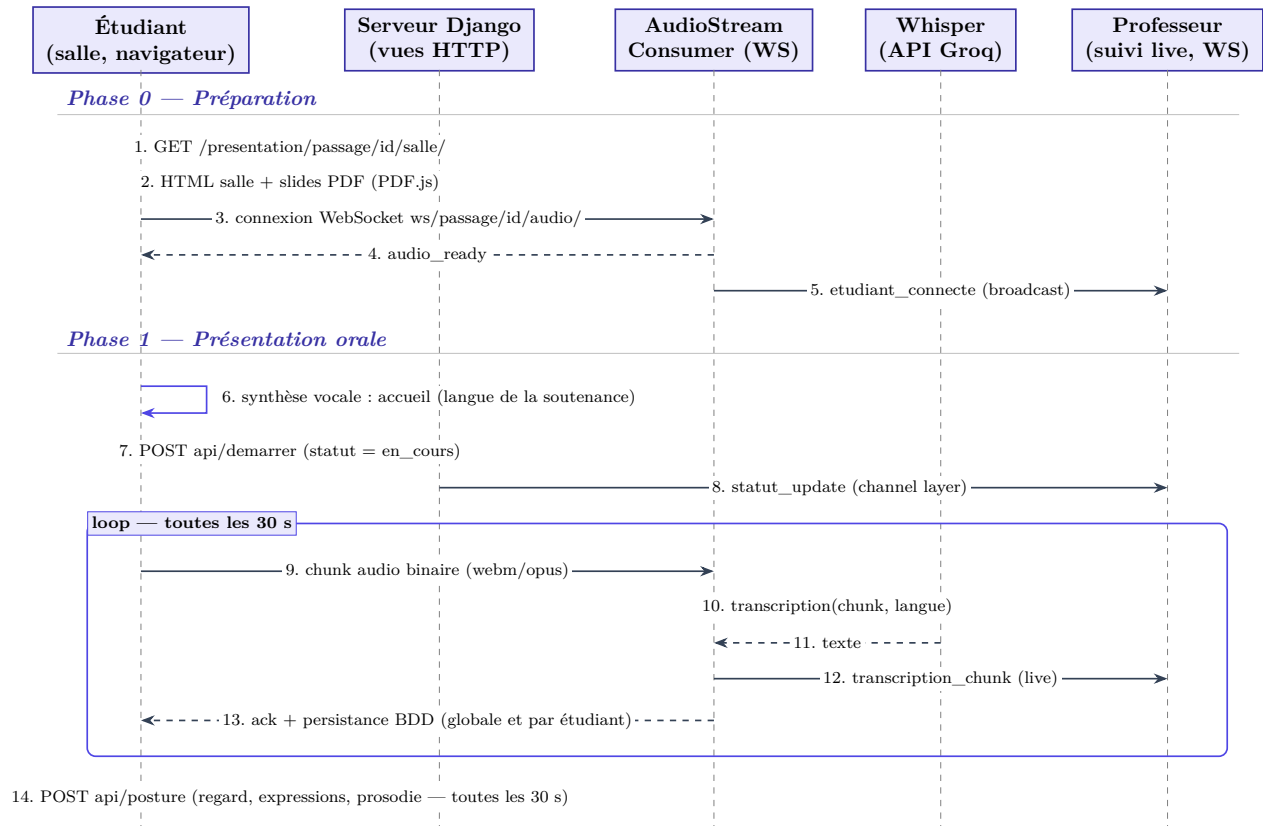


FIGURE 2.4 – Diagramme de séquence (1/2) : préparation et présentation orale avec transcription continue

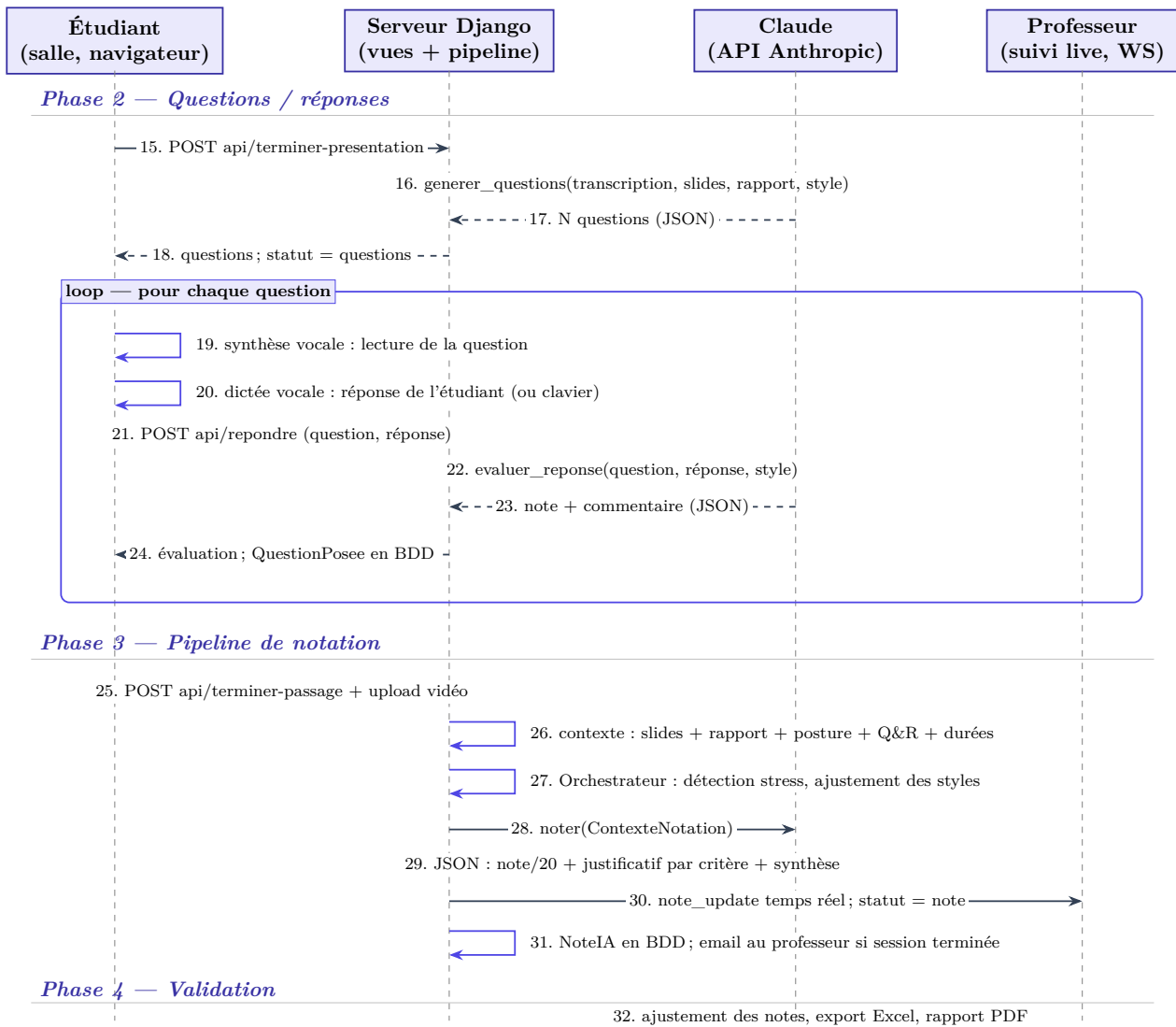


FIGURE 2.5 – Diagramme de séquence (2/2) : questions/réponses, pipeline de notation et validation

Lecture du diagramme. Trois canaux coexistent :

1. le canal **HTTP classique** (étudiant → vues Django) pour les transitions d'état et les uploads ;
2. le canal **WebSocket audio** (messages 9–13) : chaque chunk de 30 s est transcrit par Whisper puis diffusé *simultanément* au professeur (live) et persisté en base ;
3. le canal **WebSocket live** du professeur, qui ne reçoit que des diffusions de groupe (**channel layer**) : connexion étudiant, statuts, transcription, notes — et peut injecter une question qui sera lue à voix haute par la synthèse vocale de la salle.

Les interactions 6, 19 et 20 sont locales au navigateur (Web Speech API) : aucune donnée vocale de synthèse ne transite par le serveur, ce qui réduit la latence et la charge.

Chapitre 3

Choix technologiques et architecture

3.1 Django et le pattern MVT

Django implémente une variante du patron MVC appelée **MVT** (*Model-View-Template*). La correspondance est la suivante :

MVC classique	Django (MVT)	Dans SoutenanceAI
Model	Model (ORM)	<code>accounts/models.py</code> , <code>sessions_app/models.py</code> , <code>notation/models.py</code>
View (présentation)	Template	<code>templates/</code> (héritage de <code>base.html</code>)
Controller	View + URLconf	<code>views.py</code> de chaque app + <code>urls.py</code>

TABLE 3.1 – Correspondance MVC / MVT

Le « contrôleur » au sens strict est assuré par le framework lui-même : le routeur d'URL résout la requête vers une *vue*, qui orchestre modèles et templates. Cette inversion est au cœur de la productivité de Django : le développeur n'écrit que la logique métier.

Le projet est découpé en **quatre applications** à responsabilité unique :

accounts

Utilisateur personnalisé, authentification, inscription professeur, import CSV d'étudiants, réinitialisation de mot de passe, décorateurs d'autorisation.

sessions_app

Classes, soutenances, critères, passages, planification (manuelle, automatique, import), partage par code/QR, exports Excel, emails de rappel.

notation

Modèles de notes et questions, agents IA, pipeline de notation, services de traitement de fichiers, consumers WebSocket, rapports PDF.

presentation

Expérience étudiant : dashboard, dépôt de fichiers, salle de soutenance et ses APIs.

3.2 ORM, migrations et intégrité

Toutes les tables sont décrites en Python ; les 9 migrations versionnées garantissent la reproductibilité du schéma (un `User` étendu de `AbstractUser` avec un champ `role`, et les six modèles métier de la figure 2.2). Plutôt que de reproduire un modèle classique, montrons un mécanisme propre au projet : la **résolution de la transcription par étudiant** dans un passage de groupe. Chaque étudiant présente depuis son propre appareil ; l'identification faciale attribue les chunks transcrits au bon locuteur, stockés dans un champ JSON `transcriptions_par_etudiant`, avec repli sur la transcription globale :

Listing 3.1 – Résolution de la transcription individuelle (notation/pipeline.py)

```

1 def _resoudre_transcription(passage: PassageEtudiant, etudiant) -> str:
2     """Transcription individuelle si l'étudiant a présenté depuis
3     son propre appareil, sinon transcription globale du passage."""
4     trans_indiv = (passage.transcriptions_par_etudiant or {}) \
5                     .get(str(etudiant.id))
6
7     if trans_indiv:
8         return trans_indiv          # notation individuelle possible
9     return passage.transcription    # repli : transcription commune

```

L'isolation entre professeurs est obtenue par une discipline systématique dans les vues : chaque objet est récupéré avec son propriétaire dans le filtre, ce qui transforme tout accès illégitime en 404 :

Listing 3.2 – Contrôle d'accès type (extrait de sessions_app/views.py)

```

1 @professeur_required
2 def detail_session(request, session_id):
3     session = get_object_or_404(Session, pk=session_id,
4                                 professeur=request.user)
5     ...

```

3.3 Temps réel : ASGI, Channels et WebSockets

Le cycle requête/réponse HTTP ne convient ni à la transcription en direct ni au suivi live. Le projet bascule donc sur **ASGI** : Daphne sert l'application, et `ProtocolTypeRouter` aiguille HTTP vers Django classique et `ws://` vers deux consumers :

- **LivePassageConsumer** — canal d'événements d'un passage (statuts, notes, transcription, questions du professeur). Connexion refusée avec le code 4001 si non authentifié, 4003 si l'utilisateur n'est ni le professeur de la session, ni un étudiant du passage, ni superadmin.
- **AudioStreamConsumer** — canal *binaire* : reçoit les chunks audio de 30 s, les transcrit via Whisper, persiste le texte (global + par étudiant identifié) et le diffuse au groupe.

Le cœur du second consumer illustre la mécanique complète : réception d'une trame binaire, transcription asynchrone, persistance et double diffusion (groupe + accusé direct) :

Listing 3.3 – Réception d'un chunk audio (notation/consumers.py, abrégé)

```

1 class AudioStreamConsumer(AsyncWebsocketConsumer):
2
3     async def receive(self, text_data=None, bytes_data=None):
4         if text_data:                                # trame JSON (ex: face_event)
5             return await self._handle_text(text_data)
6         if not bytes_data:
7             return
8         self.chunk_counter += 1
9         n = self.chunk_counter
10        chemin = await self._sauver_chunk(self.passage_id, n, bytes_data)
11        texte = await self._transcrire(chemin, self.langue)    # Whisper/
12                                Groq
13        if not texte:
14            return await self.send(json.dumps(
15                {'type': 'audio_ack', 'chunk': n, 'transcrit': False}))
16        # Persistance : transcription globale + individuelle (locuteur
17                                identifié)

```

```

16     await self._append_transcription(self.passage_id, texte)
17     await self._append_transcription_etudiant(
18         self.passage_id, self.user_id, texte)
19     # Diffusion temps réel au groupe (le professeur la reçoit en live)
20     await self.channel_layer.group_send(self.group_name, {
21         'type': 'transcription_chunk',
22         'payload': {'texte': texte, 'chunk': n,
23                     'user_id': self.user_id, 'nom': self.user_nom},
24     })
25     await self.send(json.dumps(
26         {'type': 'audio_ack', 'chunk': n, 'transcrit': True}))

```

La diffusion s’appuie sur le *channel layer* (en mémoire en développement, Redis en production via REDIS_URL) : un événement émis par le pipeline de notation côté HTTP est instantanément reçu par tous les WebSockets du groupe `passage_<id>`. Les accès BDD depuis le contexte asynchrone passent par `database_sync_to_async`, contrainte structurante de Channels que le projet respecte systématiquement.

3.4 La couche IA

3.4.1 Transcription — Whisper sur Groq

Le modèle `whisper-large-v3` est interrogé via l’API Groq, choisie pour sa latence très faible (inférence LPU), critère décisif pour une transcription « quasi live » par tranches de 30 s, dans la langue de la soutenance.

3.4.2 Évaluation — Claude et le double axe de personnalité

L’originalité du système réside dans la séparation de la personnalité du jury en **deux axes indépendants**, chacun étant un *system prompt* distinct :

Style de questionnement (7)	Style de notation (7)
Le Mentor — questions ouvertes, guidantes	Le Généreux — $\approx 15\text{--}18/20$, valorise l’effort
Le Pédagogue — explique et enseigne	L’Indulgent — $\approx 13\text{--}16/20$, bénéficie du doute
Le Perfectionniste — traque les imprécisions	Le Juste — $\approx 12\text{--}15/20$, grille stricte
Le Contradicteur — contre-argumente	L’Avare de points — $\approx 10\text{--}13/20$
Le Stratège — questions ciblées	Le Sévère — $\approx 8\text{--}12/20$
Le Provocateur — pousse dans les re-tranchements	Le Terroriste — $\leq 10/20$, note sanction
L’Impassible — neutre, aucune réaction	Le Comptable — calcule à la décimale

TABLE 3.2 – Les $7 \times 7 = 49$ combinaisons de personnalité du jury IA

Un **orchestrateur** applique un garde-fou pédagogique : une heuristique mesure la densité de marqueurs d’hésitation dans la transcription ; si l’étudiant est jugé stressé et que le professeur a laissé l’option active, les styles agressifs basculent automatiquement vers des styles bienveillants :

Listing 3.4 – Orchestrateur anti-stress (notation/agents.py)

```

1 class Orchestrateur:
2     STYLES_AGRESSIFS_Q = {'contradicteur', 'provocateur', 'impassible'}
3     STYLES_SEVERES_N = {'severe', 'terroriste', 'avare'}
4
5     def choisir_styles(self, transcription):
6         if not self.ajustement_auto:
7             return self.sq, self.sn
8         ton = self.detecter_ton(transcription) # 'stressé' / 'hésitant'
9         / 'assuré'
10        sq, sn = self.sq, self.sn
11        if ton == 'stressé':
12            if sq in self.STYLES_AGRESSIFS_Q: sq = 'mentor'
13            if sn in self.STYLES_SEVERES_N: sn = 'indulgent'
14        return sq, sn

```

L'IA reçoit un *contexte de notation* exhaustif (transcription, texte des slides extrait par python-pptx, texte du rapport extrait par PyMuPDF, session questions/réponses, mesures comportementales agrégées, durée réelle contre durée prévue, consignes du professeur, grille pondérée) et doit répondre en **JSON strict** : pour chaque critère, une note sur 20 et un justificatif citant le texte. Ce format contraint rend la réponse directement exploitable par le pipeline et vérifiable par le professeur.

3.4.3 Analyse comportementale — dans le navigateur

Par choix de confidentialité et de performance, l'analyse visuelle reste côté client : **face-api.js** mesure le contact visuel, les 7 expressions faciales et identifie le locuteur par comparaison avec les photos de profil (essentiel pour les passages en groupe) ; la **Web Audio API** mesure volume, variation d'intonation et ratio de silence. Seuls des agrégats numériques sont envoyés au serveur, jamais le flux vidéo d'analyse.

3.5 Internationalisation et expérience utilisateur

L'interface est traduite en **cinq langues** (français source, anglais, arabe, espagnol, allemand) via le système i18n de Django (`{% trans %}`, `LocaleMiddleware`, sélecteur `set_language` présent sur toutes les pages, y compris la salle et la landing page). L'arabe bénéficie d'un support **RTL** complet (`dir="rtl"`, feuilles de style dédiées). La compilation des catalogues s'effectue par un script polib maison (`compile_messages.py`), GNU gettext n'étant pas disponible sous Windows. La langue de la *soutenance* (questions IA, synthèse vocale, transcription) est indépendante de la langue d'interface.

Côté UX : thème clair/sombre persistant avec script anti-flash, icônes vectorielles Lucide uniformes, identité visuelle indigo/violet centralisée en variables CSS, landing page marketing dédiée.

3.6 Architecture générale

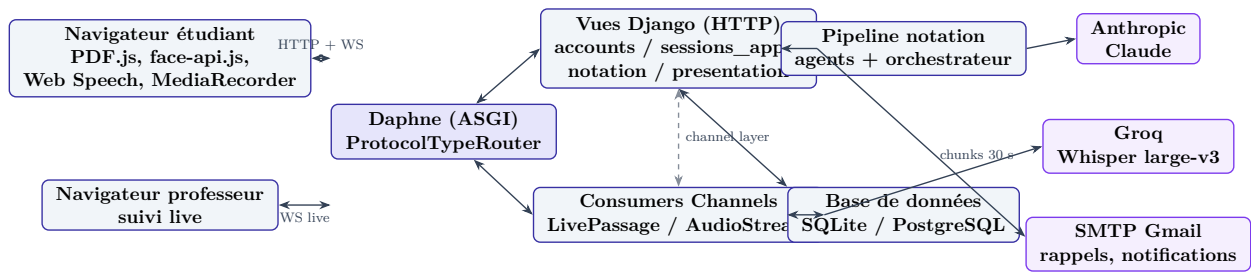


FIGURE 3.1 – Architecture générale : clients, serveur ASGI, services externes

Chapitre 4

Réalisation

4.1 Organisation du code

Listing 4.1 – Arborescence du projet (simplifiée)

```
1 soutenanceai/  
2 |-- manage.py  
3 |-- compile_messages.py          # compilation .po -> .mo (polib)  
4 |-- requirements.txt  
5 |-- soutenanceai/                # settings, urls racines, asgi, wsgi  
6 |-- accounts/                    # User, auth, roles, import CSV,  
    decorators  
7 |-- sessions_app/                # Classe, Session, Passage, planification  
    ,  
8 |   |                            # exports XLSX, emails (thread + command)  
9 |-- notation/                    # NoteIA, agents, pipeline, services,  
10 |   |                            # consumers WS, rapports PDF  
11 |-- presentation/               # dashboard etudiant, upload, salle, APIs  
12 |-- templates/                  # base.html, landing, pages par app  
13 |-- static/css|js|img           # main.css (variables, themes), avatars  
14 |-- locale/{en,ar,es,de}/       # traductions (fr = source)  
15 '-- tests/                       # 258 tests pytest
```

4.2 Authentification et parcours d'inscription

Trois parcours coexistent :

1. le **professeur** s'inscrit librement (formulaire dédié avec choix du sexe — utilisé pour l'avatar du jury dans la salle) et est connecté immédiatement ;
2. l'**étudiant** est créé *en rejoignant une classe* : le formulaire public du code d'accès crée le compte (identifiant choisi par l'étudiant, validé par `UnicodeUsernameValidator` et testé en unicité), l'inscrit à la classe et le connecte — une seule étape ;
3. le professeur peut aussi **importer un CSV** d'étudiants : les comptes sont générés en masse (identifiant `prenom.nom` normalisé ASCII, mots de passe aléatoires) et restitués dans un CSV téléchargeable.

L'autorisation est centralisée dans des décorateurs réutilisables (`accounts/decorators.py`) appliqués à chaque vue : `@professeur_required`, `@etudiant_required`, `@superadmin_required`, `@role_required(...)`.

4.3 Classes, soutenances et partage par code

Chaque `Classe` et chaque `Session` reçoit à la création un **code unique de 8 caractères** généré par tirage cryptographique (`secrets.choice`) avec re-tirage en cas de collision. La

page de partage affiche le code, le lien direct et un QR code. Le formulaire de création de soutenance combine un **ModelForm** (16 champs typés) et une grille de critères dynamique : critères prédéfinis cochables avec coefficient, plus des critères personnalisés ajoutés côté client et reconstruits côté serveur dans une transaction atomique.

4.4 Planification des passages

La page de planification réunit trois mécanismes :

- **manuel** : sélection d'étudiants non encore planifiés (le type monôme/binôme/groupe est déduit du nombre sélectionné), numéro d'ordre et heure ;
- **automatique** : à partir d'une date, d'une heure de début, d'une durée par passage et d'une pause, le serveur génère tous les passages pour les inscrits restants, en ordre alphabétique ou aléatoire, par monômes, binômes ou groupes ;
- **import de groupes** : un fichier (CSV/XLSX) de compositions de groupes est analysé, rapproché des inscrits (matching tolérant), puis confirmé par le professeur.

Un recalcul en cascade des horaires et un **export Excel** mis en forme (openpyxl) complètent l'outillage.

4.5 La salle de soutenance

La salle est une page autonome qui orchestre, côté navigateur :

- **PDF.js** pour le rendu slide par slide du PPTX converti (LibreOffice headless, repli PowerPoint COM sous Windows) ;
- la **webcam** de l'étudiant (coin bas gauche) et l'**avatar du jury IA** (coin bas droit, choisi selon le sexe du professeur) ;
- **MediaRecorder** : enregistrement vidéo complet du passage, téléversé en fin de soutenance, et chunks audio de 30 s envoyés en binaire sur le WebSocket ;
- **Web Speech API** : synthèse vocale (accueil, lecture des questions, remerciements — dans la langue de la soutenance) et reconnaissance vocale pour dicter les réponses ;
- **face-api.js** : 4 réseaux de neurones chargés dans le navigateur (détection, repères faciaux, reconnaissance, expressions) pour le contact visuel, les émotions et l'identification du locuteur par photo de profil ;
- un timer strict, la protection contre la fermeture d'onglet, et la bannière d'accueil personnalisée.

L'interface de la salle est entièrement internationalisée : libellés `{% trans %}` côté template et dictionnaire I18N injecté pour les chaînes manipulées en JavaScript.

4.6 Le pipeline de notation

Au passage à l'état **termine**, le pipeline (`notation/pipeline.py`) déroule les étapes suivantes :

1. extraction du texte des slides (`python-pptx`) et du rapport (`PyMuPDF`) ;
2. agrégation des mesures comportementales (`resumer_posture`) : % de contact visuel, émotion dominante, débit de parole interprété, ratio de silence, variation d'intonation ;

3. construction du `ContexteNotation` (dataclass) incluant la session questions/réponses intégrale et la durée réelle ;
4. passage par l'`Orchestrateur` (ajustement anti-stress éventuel), appel Claude, parsing JSON robuste (nettoyage des blocs markdown résiduels, gestion des erreurs) ;
5. persistance des `NoteIA` (une par critère — et par étudiant en mode individuel), calcul de la moyenne pondérée ;
6. le cas échéant : évaluation de la **démo vidéo** (frames extraites et envoyées à un modèle de vision) et du **dépôt GitHub** (README, structure, qualité) selon les critères du professeur ;
7. diffusion temps réel `note_update` vers le professeur et **email automatique** lorsque tous les passages de la session sont notés.

L'extrait suivant montre la colonne vertébrale du pipeline : la collecte du contexte multi-modal puis la boucle de notation par étudiant — c'est ici que convergent toutes les sources de données du projet :

Listing 4.2 – Cœur du pipeline (`notation/pipeline.py`, abrégé)

```

1 def noter_passage(passage: PassageEtudiant) -> dict:
2     session = passage.session
3     contenu_slides = extraire_texte_pptx(passage.fichier_pptx.path)
4     contenu_rapport = extraire_texte_pdf(passage.fichier_rapport.path)
5
6     # Agrégats comportementaux (face-api.js + Web Audio, calculés en série
7     )
8     posture = resumer_posture(passage.donnees_posture.get('serie', []))
9     posture = enrichir_posture_avec_prosody(
10         posture, passage.donnees_posture.get('prosody', {}))
11
12     # Durée réelle et débit de parole (mots/min) déduits des horodatages
13     duree_reelle = (passage.date_fin - passage.date_debut).total_seconds()
14     / 60
15     posture['debit_mots_par_min'] = round(
16         len(passage.transcription.split()) / duree_reelle, 1)
17
18     # Session Q&R intégrale, réinjectée dans le contexte d'évaluation
19     questions_reponses = [
20         {'question': q.question, 'reponse': q.reponse_etudiant or ''}
21         for q in QuestionPosee.objects.filter(passage=passage, auteur='ia'
22     )
23 ]
24
25 orchestrateur = Orchestrateur(
26     style_questionnement=session.style_questionnement,
27     style_notation=session.style_notation,
28     ajustement_auto=session.ajustement_auto_stress,
29 )
30
31 for etudiant in passage.etudiants.all():
32     ctx = ContexteNotation(
33         transcription=_resoudre_transcription(passage, etudiant),
34         contenu_slides=contenu_slides, contenu_rapport=contenu_rapport
35         ,
36         criteres=criteres_data, donnees_posture=posture,
37         questions_reponses=questions_reponses,
38         duree_prevue_min=session.duree_presentation,
39         duree_reelle_min=duree_reelle, ...)
40     resultat = orchestrateur.noter(ctx)          # appel Claude, JSON
41     strict

```

4.7 Suivi live et validation par le professeur

La page de suivi live affiche en colonnes la transcription qui défile, le statut, les événements (connexion de l'étudiant, questions posées) et les notes dès leur production. Le professeur peut envoyer une question : elle transite par le WebSocket, est lue à voix haute dans la salle et enregistrée comme `QuestionPosee` d'auteur « prof ».

La page de notes présente, par critère : note IA, justificatif, champ de note finale et commentaire du professeur. La moyenne pondérée est recalculée à la volée. Deux exports : **Excel** (toutes les notes de la session) et **rapport PDF individuel** (ReportLab) reprenant l'identité du candidat, la grille, les justificatifs, la synthèse comportementale et la transcription.

4.8 Emails automatiques

Deux emails transactionnels (SMTP Gmail, mot de passe d'application en variable d'environnement) :

- **rappel à J-10 minutes** pour chaque étudiant d'un passage (champ `rappel_envoye` anti-doublon). L'envoi est porté par un *thread démon* démarré avec le serveur (vérification chaque minute d'une fenêtre de 9 à 11 minutes avant l'heure prévue), avec une management command de secours `envoyer_rappels` pour un déclenchement par planificateur externe ;
- **notification de fin de session** au professeur quand le dernier passage est noté.

Chapitre 5

Tests, sécurité et qualité

5.1 Stratégie de tests

La suite compte **258 tests** (pytest, pytest-django, pytest-asyncio), exécutables sans aucune clé API grâce au mocking systématique des services externes :

Module	Ce qui est vérifié
test_models	Génération et unicité des codes, valeurs par défaut, statuts, relations
test_views_*	Codes HTTP, contrôle d'accès par rôle, isolation entre professeurs (accès croisé $\Rightarrow$ 404/403), formulaires valides et invalides
test_agents	7+7 styles définis, prompts combinés, parsing JSON, gestion d'erreurs API, bascules de l'orchestrateur (stress / calme / option désactivée)
test_pipeline	Notes persistées par critère, moyenne pondérée, durée réelle
test_consumers	Connexions WS acceptées/refusées (4001, 4003), diffusion de la transcription au groupe, robustesse aux trames invalides
test_services	Transcription mockée, extraction PPTX/PDF réelles sur fichiers générés, agrégats de posture

TABLE 5.1 – Cartographie de la suite de tests

Trois exemples illustrent les trois niveaux de difficulté de la suite. D'abord le test de sécurité le plus rentable du projet — l'isolation entre professeurs, qui matérialise un risque métier réel :

Listing 5.1 – Niveau 1 : isolation entre professeurs

```
1 def test_prof_cannot_see_other_profs_session(self, client_prof, prof_user2, db):
2     s2 = Session.objects.create(professeur=prof_user2, titre='Pas la mienne',
3                                 langue='fr', duree_presentation=10)
4     resp = client_prof.get(reverse('sessions_app:detail_session', args=[s2.pk]))
5     assert resp.status_code == 404
```

Ensuite, le test des agents IA *sans aucune clé API* : le client Anthropic est remplacé par un mock dont on contrôle la réponse, ce qui permet de vérifier le parsing JSON, la propagation des styles dans le system prompt et la robustesse aux réponses malformées :

Listing 5.2 – Niveau 2 : mock de l'API Claude (tests/test_agents.py)

```
1 @patch('notation.agents.get_client')
```

```

2 def test_noter_styles_used_in_system_prompt(self, mock_get_client):
3     mock_client = MagicMock()
4     mock_client.messages.create.return_value = _make_claude_response(
5         json.dumps({'notes': [], 'synthese': '', 'ton_detecte': 'calme'}))
6     mock_get_client.return_value = mock_client
7
8     agent = AgentNotation('perfectionniste', 'severe')
9     agent.noter(_make_context())
10
11     # Le system prompt envoyé à Claude combine bien les DEUX axes choisis
12     system_used = mock_client.messages.create.call_args[1]['system']
13     assert QUESTIONNEMENT_PROMPTS['perfectionniste'][:20] in system_used
14     assert NOTATION_PROMPTS['severe'][:20] in system_used

```

Enfin, le niveau le plus technique : tester un **consumer WebSocket asynchrone**. Le consumer est monté dans un **URLRouter** de test (pour peupler `scope['url_route']` comme en production), l'utilisateur est injecté dans le scope, et l'on vérifie le refus d'accès avec le bon code de fermeture :

Listing 5.3 – Niveau 3 : consumer WebSocket asynchrone (`tests/test_consumers.py`)

```

1 def make_ws_app_with_user(consumer_class, user=None):
2     router = URLRouter([re_path(
3         r'^ws/passage/(?P<passage_id>\d+)/(?:audio/)?$',
4         consumer_class.as_asgi())])
5     async def auth_inject(scope, receive, send):
6         if user is not None:
7             scope['user'] = user
8             await router(scope, receive, send)
9         return auth_inject
10
11 @pytest.mark.django_db(transaction=True)
12 @pytest.mark.asyncio
13 async def test_unauthorized_user_closes_4003(self, passage_obj, prof_user2):
14     """Un professeur étranger au passage est rejeté (code 4003)."""
15     app = ws_app(LivePassageConsumer, user=prof_user2)
16     communicator = WebsocketCommunicator(app, f'/ws/passage/{passage_obj.pk}/')
17     connected, code = await communicator.connect()
18     assert not connected
19     assert code == 4003

```

La suite couvre également la diffusion de bout en bout : un test connecte *deux* communicateurs (étudiant sur le canal audio, professeur sur le canal live), envoie un chunk binaire avec Whisper mocké, et vérifie que le professeur reçoit l'événement `transcription_chunk` — soit le parcours complet des messages 9 à 12 de la figure 2.4.

5.2 Sécurité

- **Secrets** : toutes les clés (Anthropic, Groq, SMTP, SECRET_KEY) vivent dans un fichier `.env` exclu du dépôt Git ; un `.env.example` documenté est fourni.
- **Contrôle d'accès** : décorateur de rôle sur chaque vue + filtrage par propriétaire dans chaque requête ORM ; vérification d'appartenance au passage pour chaque endpoint de la salle.

- **WebSockets** : authentification et autorisation à la connexion (codes 4001/4003), vérifiées par les tests.
- **CSRF** actif globalement ; **validation** des identifiants choisis par les étudiants (caractères autorisés, unicité).
- **Confidentialité** : l'analyse faciale s'exécute dans le navigateur ; seuls des agrégats chiffrés (pourcentages, moyennes) atteignent le serveur.

5.3 Qualité et maintenabilité

- Identité visuelle centralisée en **variables CSS** (palette indigo/violet unique, thèmes clair/sombre) ;
- logique IA isolée dans des **services purs** sans dépendance aux vues, donc testable unitairement ;
- conventions strictes : aucun emoji dans le code (icônes Lucide), terminologie unifiée (« soutenance » côté interface, **Session** côté modèle), messages utilisateur traduits.

Conclusion et perspectives

SoutenanceAI démontre qu'un framework de prototypage web comme Django permet, en un module universitaire, de construire bien plus qu'un CRUD : une application temps réel complète où cohabitent HTTP classique, WebSockets, services d'IA générative et traitement multimédia, le tout sous contrôle d'une suite de tests substantielle.

Les objectifs fixés au chapitre 1 sont atteints : gestion complète côté professeur (O1), salle immersive côté étudiant (O2), notation justifiée et ajustable (O3), cinq langues avec RTL (O4), qualité vérifiable — 258 tests, contrôle d'accès systématique, secrets externalisés (O5).

Bilan critique : ce que le projet ne fait pas (encore) bien

Un bilan honnête impose de nommer les limites et les compromis assumés en l'état actuel du prototype.

- **Le pipeline de notation est synchrone et bloquant.** L'appel à Claude (contexte volumineux) prend de l'ordre de 30 à 60 secondes pendant lesquelles la requête HTTP reste ouverte. Acceptable pour un passage isolé, inacceptable en charge : c'est la dette technique n° 1, identifiée dès la conception (d'où l'isolation du pipeline dans un module sans dépendance aux vues, prêt à être déplacé derrière Celery sans réécriture).
- **La détection de stress est une heuristique simpliste.** Elle compte la densité d'une liste *fixe et codée en dur* de marqueurs d'hésitation (« euh », « heu », « je ne sais pas », points de suspension...) rapportée au nombre de mots, avec deux seuils (2 % et 5 %). Elle n'est ni configurable par le professeur, ni validée linguistiquement, ni adaptée aux cinq langues (les marqueurs sont francophones). Les mesures comportementales (expressions faciales, prosodie) ne participent pas encore à cette décision alors qu'elles sont déjà collectées — c'est une incohérence interne du système, connue et documentée.
- **Les 49 combinaisons de personnalité ne sont pas validées empiriquement.** Les tests garantissent que chaque style injecte bien son prompt (vérifié par mock), mais aucune donnée ne prouve à ce jour que « Terroriste » note significativement plus bas que « Généreux » à prestation égale, ni que l'écart est stable d'un appel à l'autre. Un protocole existe (corpus de transcriptions fixes × 49 combinaisons × n répétitions, analyse de variance) mais n'a pas été exécuté, faute de budget d'API — c'est la première perspective de la section suivante.
- **La conversion PPTX → PDF n'a pas été mesurée en charge.** LibreOffice headless traite un fichier en 5 à 15 secondes sur la machine de développement, séquentiellement. Vingt étudiants qui déposent simultanément créeraient une file d'attente significative ; la parade actuelle (page de salle qui se rafraîchit pendant la conversion) masque la latence mais ne la réduit pas.
- **Le channel layer en mémoire interdit le multi-processus.** En l'état, un seul worker Daphne peut servir les WebSockets ; la bascule Redis est prévue par configuration mais n'a pas été éprouvée.
- **Aucun test d'interface de bout en bout.** Les 258 tests couvrent modèles, vues, agents, pipeline et consumers, mais pas le JavaScript de la salle (PDF.js, Web Speech,

face-api.js), testé uniquement à la main. Un crash de la dictée vocale côté navigateur ne serait détecté par aucun test automatisé.

Le projet ouvre des perspectives claires :

- passage du pipeline de notation derrière **Celery** + **Redis** pour la montée en charge ;
- streaming audio continu vers Whisper (WebSocket bout en bout) plutôt que chunks de 30 s ;
- benchmark quantitatif de la fiabilité de la notation IA contre des jurys humains (corrélation inter-juges), prolongement naturel vers une publication scientifique ;
- déploiement de production (Daphne + Nginx + PostgreSQL + Redis) sur un hébergeur à processus persistants.

Au-delà de la technique, le projet assume un positionnement pédagogique : l'IA propose, justifie et trace ; *le professeur décide*. C'est cette articulation — automatiser la mécanique pour rendre du temps au jugement humain — qui constitue, à notre sens, la véritable contribution de SoutenanceAI.

Webographie

- Documentation Django 4.2 — <https://docs.djangoproject.com/fr/4.2/>
- Django Channels — <https://channels.readthedocs.io/>
- API Anthropic (Claude) — <https://docs.anthropic.com/>
- API Groq (Whisper) — <https://console.groq.com/docs/>
- face-api.js — <https://github.com/vladmandic/face-api>
- PDF.js (Mozilla) — <https://mozilla.github.io/pdf.js/>
- ReportLab — <https://www.reportlab.com/documentation/>
- PyMuPDF — <https://pymupdf.readthedocs.io/>
- python-pptx — <https://python-pptx.readthedocs.io/>
- Lucide Icons — <https://lucide.dev/>
- Web Speech API (MDN) — https://developer.mozilla.org/fr/docs/Web/API/Web_Speech_API

Annexe : captures d'écran de l'application

Les captures suivantes sont prises sur l'application en fonctionnement (données de démonstration). Elles illustrent le parcours des deux acteurs principaux.

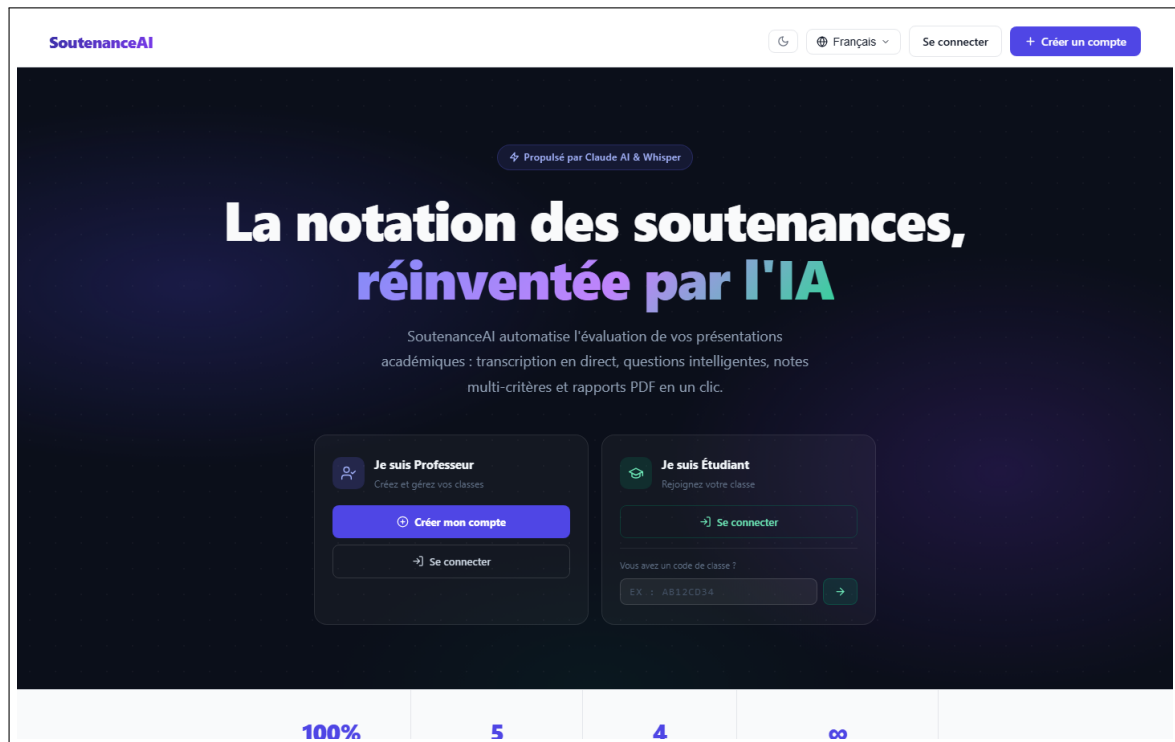


FIGURE 5.1 – Page d'accueil publique : parcours professeur et étudiant, code de classe, sélecteur de langue et thème

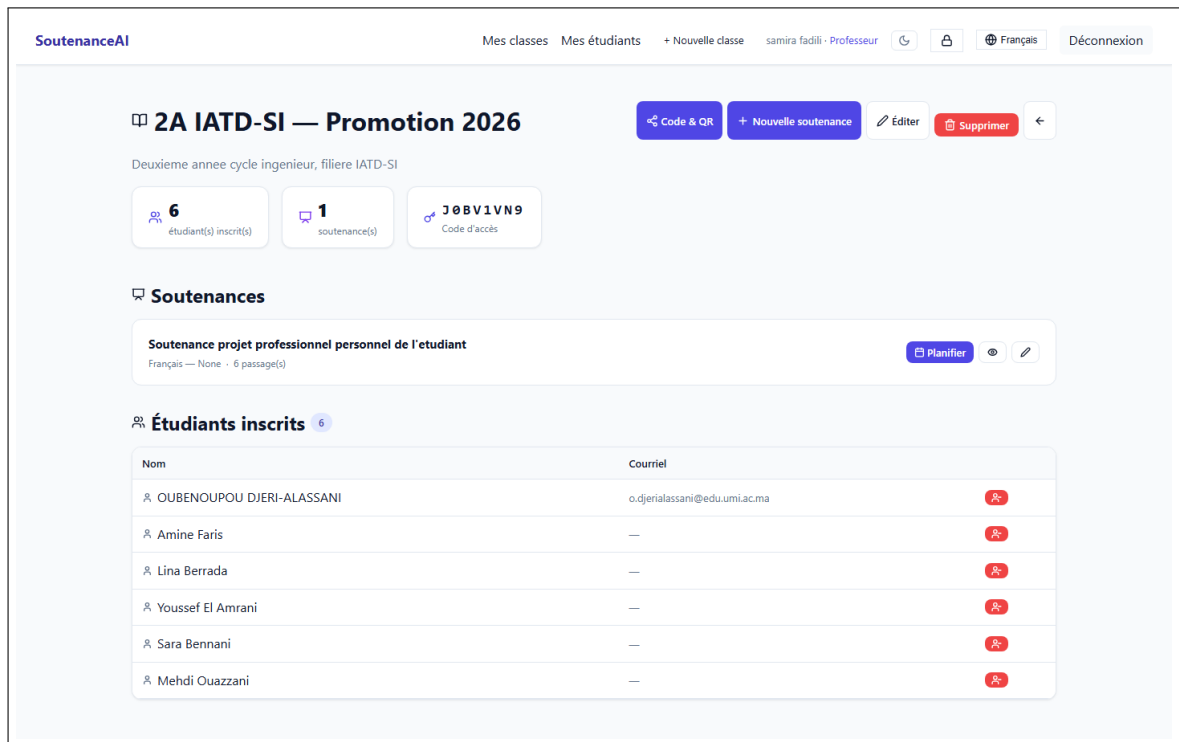


FIGURE 5.4 – Détail d'une classe : étudiants inscrits et soutenances

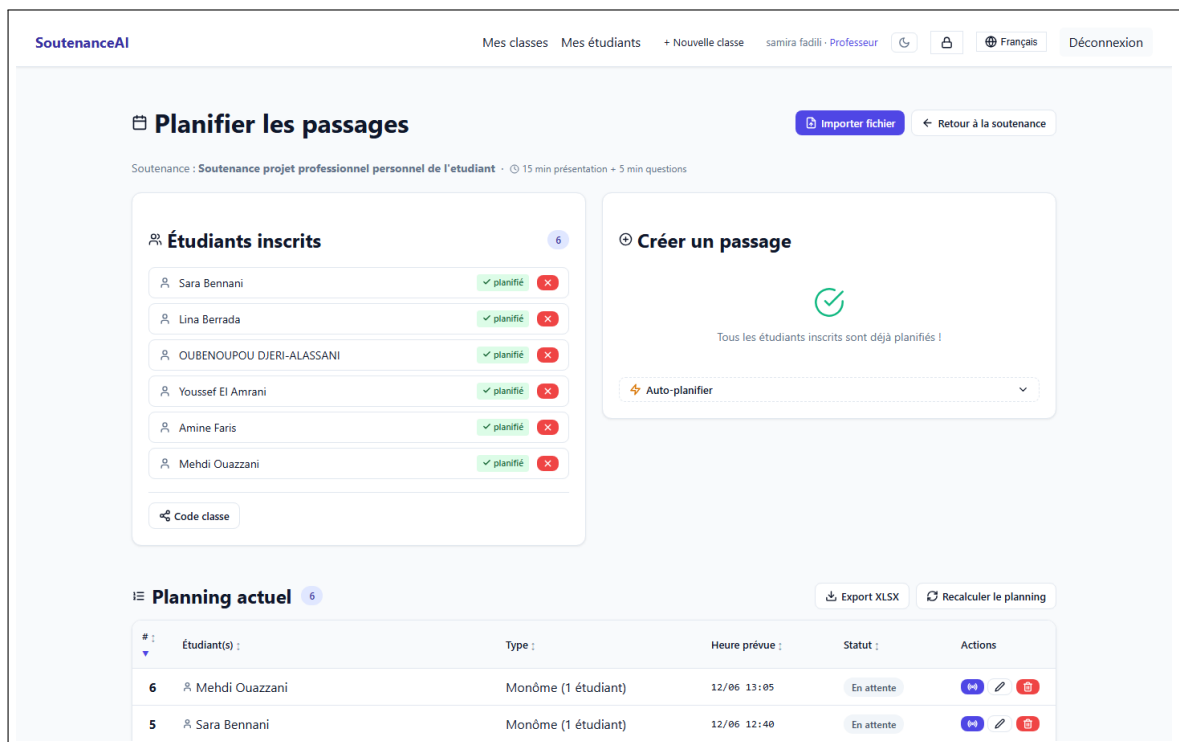


FIGURE 5.5 – Planification des passages : inscrits, création manuelle, auto-planification et planning exportable

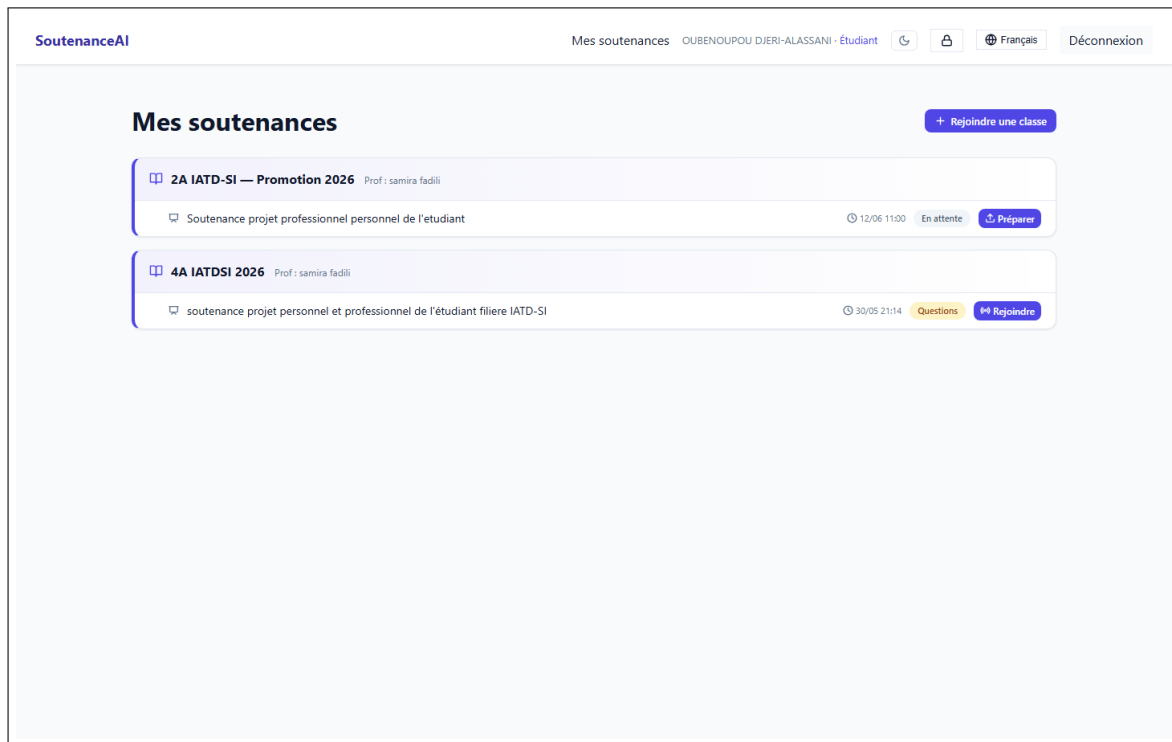


FIGURE 5.6 – Dashboard étudiant : classes, soutenances et état du passage

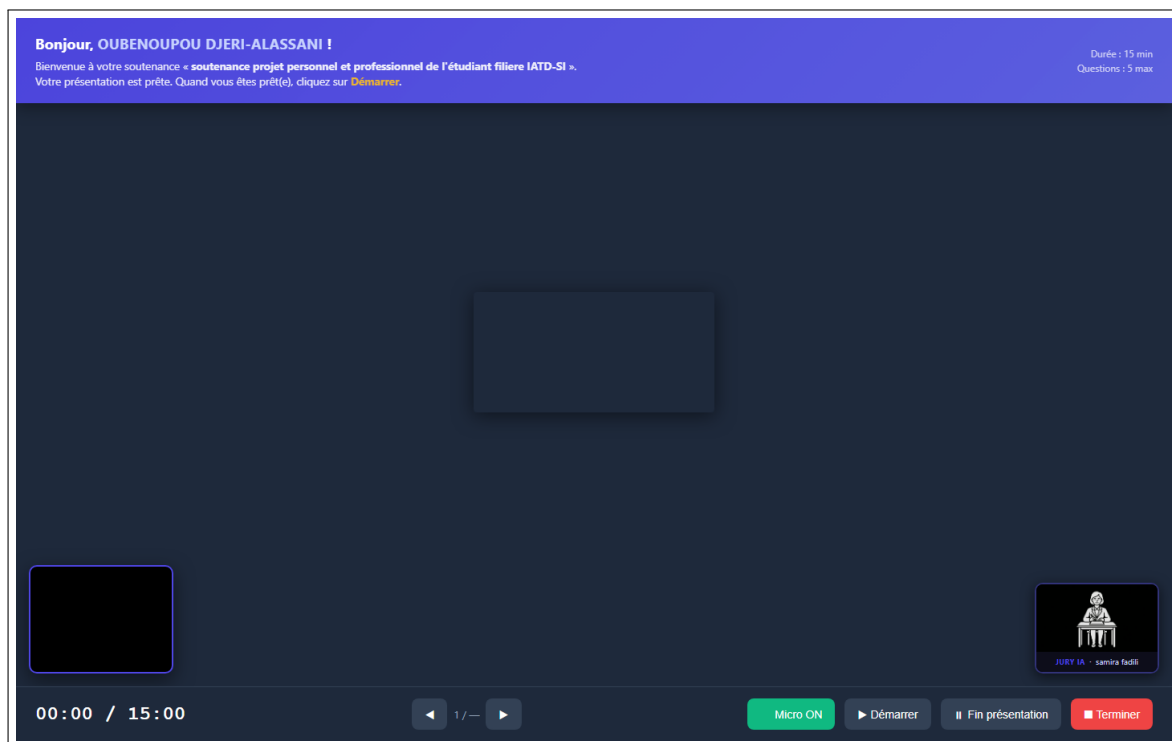


FIGURE 5.7 – Salle de soutenance : slides (PDF.js), webcam étudiant, avatar du jury IA, timer et contrôles